



Solution Design Document



Platforms	Web / Mobile Web
Products	▪ Consumer Payment Experience: PayPal Wallet
Revision History	▪ v1, 12th Sep 2026, Kenneth Suralta (ksuralta@paypal.com)
Introduction	PayPal via Braintree integration

Legal Notice

© 2026 PayPal, Inc. All rights reserved. PayPal is a registered trademark of PayPal, Inc. The PayPal logo is a trademark of PayPal, Inc. Other trademarks and brands are the property of their respective owners. The information in this document belongs to PayPal, Inc. It may not be used, reproduced or disclosed without the written approval of PayPal, Inc. Notice of non-liability: PayPal, Inc. is providing the information in this document to you "AS-IS" with all faults. PayPal, Inc. makes no warranties of any kind (whether express, implied or statutory) with respect to the information contained herein. PayPal, Inc. assumes no liability for damages (whether direct or indirect), caused by errors or omissions, or resulting from the use of this document or the information contained in this document or resulting from the application or use of the product or service described herein. PayPal, Inc. reserves the right to make changes to any information herein without further notice.

[Table of contents](#)

[1. Introduction](#)

[1.1. About This Document](#)

[2. Solution Overview](#)

[2.1. Executive Summary](#)

[2.2. Solution Scope & Requirements](#)

[3. Consumer Payment Experience](#)

[3.1. PayPal Wallet](#)

[3.1.1. PayPal Checkout from the Product/Cart Page](#)

[3.1.2. PayPal Checkout from the Payment Page](#)

[3.1.3. Create a Sandbox Account](#)

[3.1.4. Sign up for a PayPal Business Account](#)

[3.1.5. Checkout with PayPal Flow](#)

[3.1.5.1. Integration Overview](#)

[3.1.5.2. Client-side Integration](#)

[3.1.5.2.1. Web \(JS SDK\)](#)

[3.1.5.3. Server-side Integration](#)

[4. Appendix](#)

[4.1. Appendix 1: SDKs and documentation reference](#)

[4.1.1. Braintree Client/Server SDK](#)

[4.2. Appendix 2: Integration Process](#)

[4.2.1. Sandbox Setup and Testing](#)

1. Introduction

1.1. About This Document

This document outlines the solution agreed between PayPal and Campion. All sections in this document are considered in scope and will be delivered via API/SDKs unless otherwise stated in the section.

The Solution Design Document is experience-focused so that it can be shared and reviewed by all Campion teams required as part of this project. Each sub-section links to the relevant PayPal Developer webpage for additional low-level detail if required.

If there are any solution details that are currently under discussion with Campion, they will be highlighted in this document.

The SDD is structured as follows:

- **Solution Overview:** This section defines the purpose, goal, and scope of the integration
- **Consumer Payment Experience:** This section details the payment method(s) to be enabled on Campion's properties
- **Account Structure:** This section describes the account structure planned for Campion
- **Technical & Operational Payment Experience:** This section describes the technical setup, processes and back-end requirements for payment processing management.

Solution details that are currently under discussion with Campion are highlighted throughout this document.

2. Solution Overview

2.1. Executive Summary

Campion is integrating with PayPal via the Braintree platform to establish payment acceptance capabilities. PayPal will be enabled as the initial payment method, with PayPal Pay Later offered to give customers the flexibility to split purchases into manageable installments at checkout.

The integration will be custom-built, with PayPal as the sole payment method enabled at launch. This solution design outlines the technical approach for onboarding Campion to PayPal (including Pay Later) eligibility and presentment

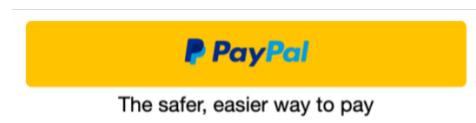
logic, and the expected customer experience. Additional payment methods may be considered for future phases.

2.2. Solution Scope & Requirements

Merchant of Record	Campion
Payment Methods	PayPal, PayPal Pay Later
Presentment Currencies	AUD
Settlement Currencies	AUD
Client-side platforms	Web, mWeb
Client-side technologies	JS v6 SDK
Server-side technologies	Braintree SDK (.NET)
Capture Model	Sale
Vaulting	No
Risk Management Tools	Data Collector
Dispute Management	Manual/API

3. Consumer Payment Experience

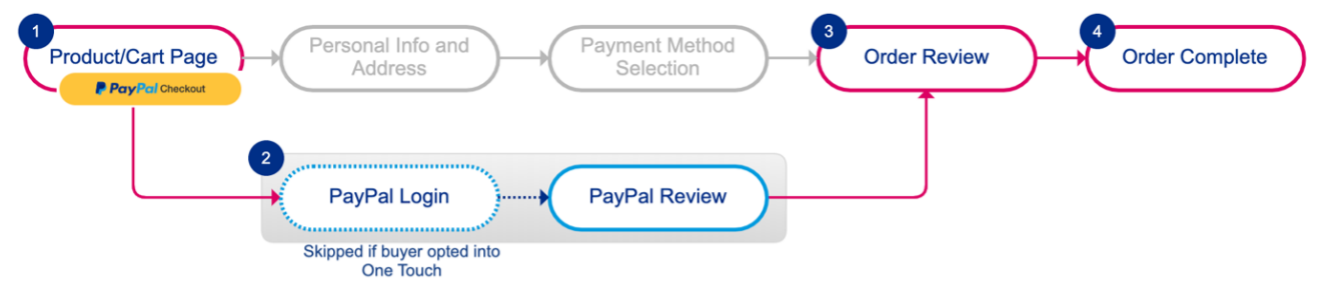
3.1. PayPal Wallet



PayPal allows customers to skip online forms and complete their order in just a few clicks, enabling Campion to boost conversion on their checkout. On web, the PayPal button is rendered by PayPal and Braintree SDKs. Campion can customise the styling of the button to align with their brand and UI guidelines.

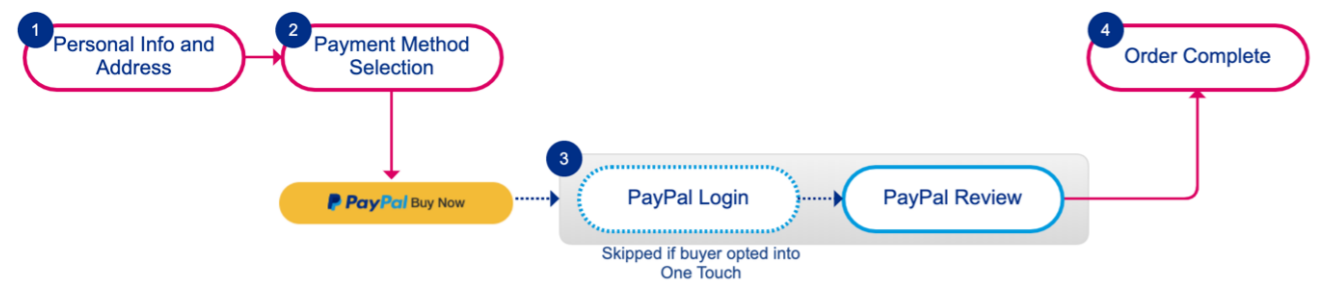
The following sections demonstrate the customer journey with PayPal placed in multiple locations across your website.

3.1.1. PayPal Checkout from the Product/Cart Page



This flow streamlines checkout by allowing customers to complete a purchase without filling in extra fields such as name, email, billing address, and other details. On clicking the PayPal Checkout button, the customer is immediately redirected to PayPal to review transaction details. The customer is then returned to Campion and their information is returned by PayPal to pre-populate the final confirmation page.

3.1.2. PayPal Checkout from the Payment Page



This flow is commenced towards the end of the checkout process, where PayPal is presented alongside other payment options, typically credit and debit cards. The customer selects PayPal and is redirected to the PayPal site to log in and confirm the purchase. For European shops, we recommend using the “PayPal Buy Now” button to provide a clear message to your customers. The total PayPal experience must be in parity with other payment methods.

3.1.3. Create a Sandbox Account

The Braintree sandbox allows you to set up a test environment to run transactions using test credit card numbers and explore the functionality of the Braintree gateway.

Sign up for a Braintree Sandbox Account at <https://www.braintreepayments.com/sandbox>.

For more details, please refer to the official documentation at: <https://developer.paypal.com/braintree/docs/start/overview#a-sandbox-account>

3.1.4. Sign up for a PayPal Business Account

In order to use PayPal with Braintree, you'll need a PayPal Business Account. You can either sign up for a new account on [PayPal's website](#), or upgrade an existing PayPal Premier account by completing the following steps:

1. Go to the [Settings page](#) in your PayPal account
2. Click **Upgrade to a Business account**
3. Enter your business information and follow the remaining prompts

For more details, please refer to the official documentation at: <https://developer.paypal.com/braintree/articles/guides/payment-methods/paypal/setup-guide#sign-up-for-a-paypal-business-account>

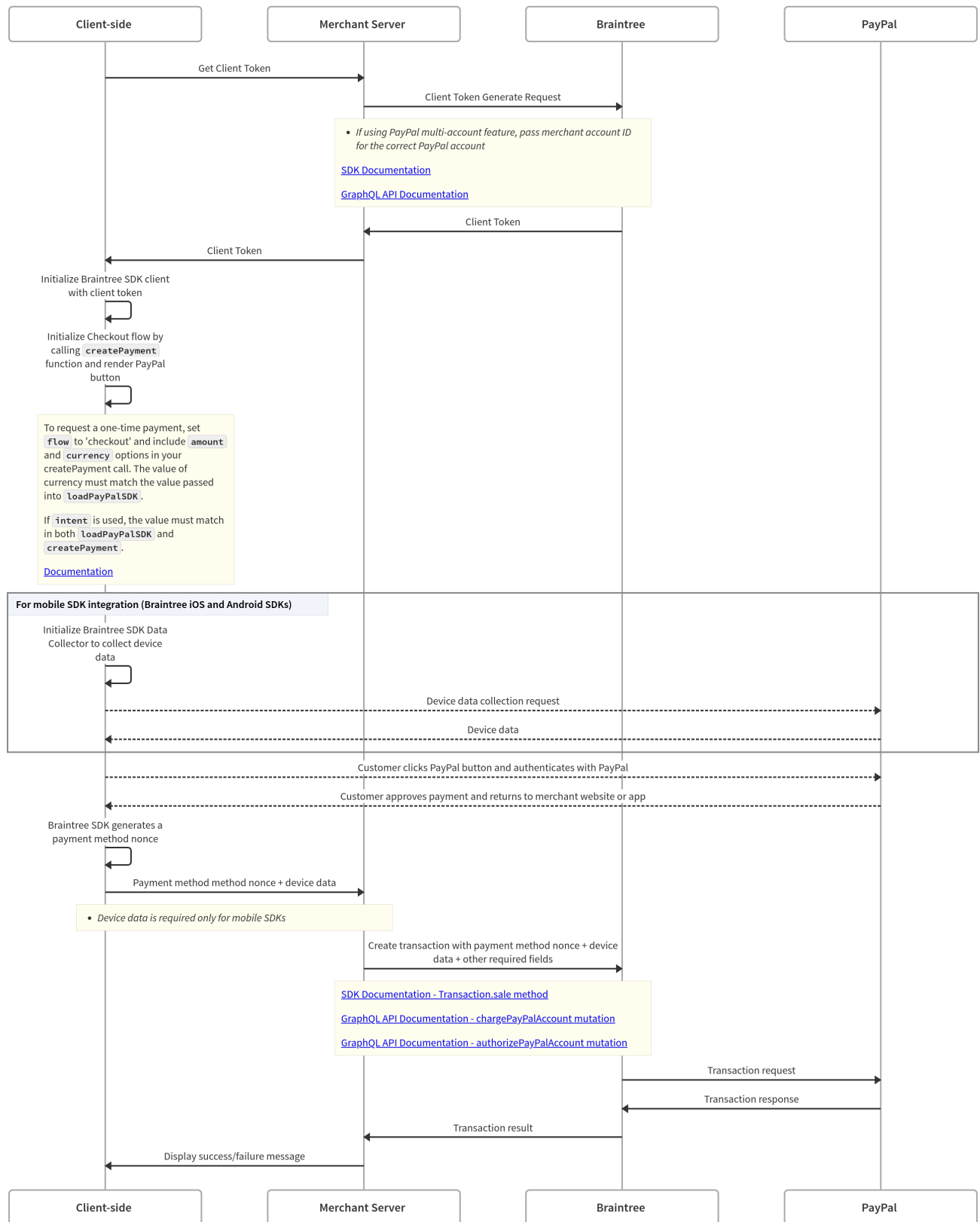
3.1.5. Checkout with PayPal Flow

Checkout with PayPal is a one-time payment checkout experience that gives you more control over the entire checkout process. It offers a streamlined checkout flow that keeps customers local to your website during the payment authorization process.

Unlike [the Vault flow](#), Checkout with PayPal does not provide the ability to store a customer's PayPal account in the Vault.

3.1.5.1. Integration Overview

Braintree Checkout with PayPal Flow



MADE WITH swimlanes.io

3.1.5.2. Client-side Integration

3.1.5.2.1. Web (JS SDK)

Before you can add a PayPal button, make sure you have a few prerequisites out of the way:

1. [Create, verify, and link your PayPal account](#) in the Braintree Control Panel
2. [Generate a client token](#) on your server

Basic configuration

Load the required JS libraries in your payment page:

```
<!-- Load the client component.-->
<script src="https://js.braintreegateway.com/web/3.142.0/js/client.min.js"></script>
<script
  async
  src="https://js.braintreegateway.com/web/3.142.0/js/paypal-checkout-v6.min.js"
  onload="onPayPalCheckoutV6Loaded()"
></script>
```

Create a div element. This is where your PayPal buttons will appear.

For the standard PayPal button:

```
<paypal-button id="paypal-button" type="pay" hidden></ paypal-button>
```

For the PayPal Pay Later button

```
<paypal-pay-later-button id="paylater-button" hidden></paypal-pay-later-button >
```

Initialize components

Every integration requires a client. Once you've created one, you can pass it to the PayPal Checkout component to start the PayPal flow.

```
async function onPayPalCheckoutV6Loaded() {
  try {
    const braintreeClientToken = await getBraintreeBrowserSafeClientToken();
    const braintreeInstance = await window.braintree.client.create({
      authorization: braintreeClientToken,
    });
```

```

const paypalCheckoutV6Instance =
  await window.braintree.paypalCheckoutV6.create({
    client: braintreeInstance,
  });

await paypalCheckoutV6Instance.loadPayPalSDK();

} catch (error) {
  renderAlert({ type: "danger", message: "Failed to initialize PayPal" });
  console.error(error);
}
}

function renderAlert({ type, message }) {
  const alertComponentElement = document.querySelector("alert-component");
  if (!alertComponentElement) {
    return;
  }

  alertComponentElement.setAttribute("type", type);
  alertComponentElement.innerText = message;
}

```

Load the PayPal JS SDK

Once the Braintree PayPal Checkout component is created, use the `loadPayPalSDK` method to load the PayPal JS SDK script file onto the page.

```
paypalCheckoutV6Instance.loadPayPalSDK();
```

Invoking the Checkout with PayPal flow

To request a one-time payment, set flow to 'checkout' and include amount and currency options in your `createOneTimePaymentSession` call.

```

async function onPayPalCheckoutV6Loaded() {
  try {
    const braintreeClientToken = await getBraintreeBrowserSafeClientToken();
    const braintreeInstance = await window.braintree.client.create({
      authorization: braintreeClientToken,

```

```

    });

    const paypalCheckoutV6Instance =
      await window.braintree.paypalCheckoutV6.create({
        client: braintreeInstance,
      });

    await paypalCheckoutV6Instance.loadPayPalSDK();

    await setupSmartPaymentStack(paypalCheckoutV6Instance);
  } catch (error) {
    renderAlert({
      type: "danger",
      message: "Failed to initialize PayPal Smart Payment Stack",
    });
    console.error(error);
  }
}

async function setupSmartPaymentStack(paypalCheckoutV6Instance) {
  const amount = "100.00";
  const currency = "AUD";

  const callbacks = {
    async onApprove(data) {
      console.log("onApprove", data);
      const { nonce } = await paypalCheckoutV6Instance.tokenizePayment(data);
      const orderData = await completePayment(nonce, amount);
      renderAlert({
        type: "success",
        message: `Order successfully captured: ${JSON.stringify(data)}`,
      });
      console.log("Capture result", orderData);
    },
    onCancel(data) {
      renderAlert({
        type: "warning",
        message: `onCancel() callback called: ${data.orderId ?? ""}`,
      });
    },
  };
}

```

```

    });
    console.log("onCancel", data);
  },
  onError(error) {
    renderAlert({
      type: "danger",
      message: `onError() callback called: ${error.message}`,
    });
    console.log("onError", error);
  },
});

// Check eligibility for all payment methods
const eligibility = await paypalCheckoutV6Instance.findEligibleMethods({
  amount,
  currency,
});

// Set up PayPal button (always available)
const paypalButton = document.querySelector("#paypal-button");
const paypalSession = paypalCheckoutV6Instance.createOneTimePaymentSession({
  amount,
  currency,
  intent: "capture",
  ...callbacks,
});
paypalButton.removeAttribute("hidden");
paypalButton.addEventListener("click", () => paypalSession.start());

// Set up Pay Later button if eligible
if (eligibility.paylater) {
  const details = eligibility.getDetails("paylater");
  const paylaterButton = document.querySelector("#paylater-button");
  paylaterButton.setAttribute("countryCode", details.countryCode);
  paylaterButton.setAttribute("productCode", details.productCode);
  const paylaterSession = paypalCheckoutV6Instance.createPayLaterSession({
    amount,
    currency,
  });
}

```

```

        intent: "capture",
        ...callbacks,
    });
    paylaterButton.removeAttribute("hidden");
    paylaterButton.addEventListener("click", () => paylaterSession.start());
}
}

```

```

async function getBraintreeBrowserSafeClientToken() {
    const response = await fetch(
        "/braintree-api/auth/browser-safe-client-token",
        {
            method: "GET",
            headers: {
                "Content-Type": "application/json",
            },
        },
    );
    const { clientToken } = await response.json();

    return clientToken;
}

```

```

async function completePayment(paymentMethodNonce, amount) {
    const response = await fetch("/braintree-api/transaction/sale", {
        method: "POST",
        headers: {
            "Content-Type": "application/json",
        },
        body: JSON.stringify({
            paymentMethodNonce,
            amount,
        }),
    });
    const result = await response.json();

    return result;
}

```

```
function renderAlert({ type, message }) {  
  const alertComponentElement = document.querySelector("alert-component");  
  if (!alertComponentElement) {  
    return;  
  }  
  
  alertComponentElement.setAttribute("type", type);  
  alertComponentElement.innerText = message;  
}
```

Use the `paypalCheckoutInstance` in the `onApprove` function of the PayPal JS SDK setup method to tokenize the PayPal account. After the customer completes the consent flow and the PayPal pop-up closes, successful tokenization will return a payment method nonce.

Send the nonce to your server and use a Braintree server SDK to call `Transaction.sale` method to create a transaction.

Sample Code: <https://github.com/paypal-examples/v6-web-sdk-with-braintree-sdk-sample-integration/tree/main/client/paypalOneTimePayments/smartStack>

3.1.5.3. Server-side Integration

Transaction Creation

Once the customer has successfully authenticated with PayPal, include the `payment_method_nonce` parameter in the `Transaction: Sale` call on your server.

Example code

snippet: <https://developer.paypal.com/braintree/docs/guides/paypal/server-side/>

Currency support

The customer will be charged in the currency associated with the `merchant_account_id` passed in the `Transaction.sale` call. We support all currencies that [PayPal REST APIs support](#).

For details on accepting foreign currencies with PayPal, see [our PayPal account setup guide](#).

Shipping addresses

If you've collected a shipping address, you will need to pass that along with the Transaction.sale call. The following fields are required when passing a shipping address for PayPal transactions:

- first_name
- last_name
- street_address
- locality
- postal_code
- region
- country_code_alpha2

For details on how to add a shipping address when creating a transaction, see [the transaction reference full example](#). PayPal validates the shipping address, so it must follow [the PayPal address conventions](#).

4. Appendix

4.1. Appendix 1: SDKs and documentation reference

4.1.1. Braintree Client/Server SDK

Getting started with Braintree:

<https://developers.braintreepayments.com/start/overview>

SDK & Code / Object References:

- Client-side Javascript SDK Information:
 - <https://developers.braintreepayments.com/start/hello-client/javascript/v3>
 - PayPal Checkout Component <https://braintree.github.io/braintree-web/current/PayPalCheckout.html>
- Client-side iOS and Android SDK Information:
 - iOS: <https://developer.paypal.com/braintree/docs/start/hello-client/ios/v6>
 - Android: <https://developer.paypal.com/braintree/docs/start/hello-client/android/v5>
- Server-side Braintree API Information:
 - <https://developers.braintreepayments.com/start/hello-server>
 - <https://graphql.braintreepayments.com/guides/>

- Repositories
 - JS: <https://braintree.github.io/braintree-web/current/PayPal.html>
 - iOS: https://braintree.github.io/braintree_ios/
 - Android: <https://www.javadoc.io/doc/com.braintreepayments.api/braintree/latest/index.html>
- Error Handling
 - Result objects & errors: <https://developers.braintreepayments.com/reference/general/result-objects/#error-results>
 - https://graphql.braintreepayments.com/guides/making_api_calls/#understanding-responses

Note: You can select your client/server languages in the top-center to dynamically change the front end and back end languages.

4.2. Appendix 2: Integration Process

4.2.1. Sandbox Setup and Testing

Prior to going live, you have the opportunity to perform testing within the Braintree Sandbox environment. Please keep in mind that the Sandbox Environment is completely separate from the production environment.

[Sign up for Braintree Sandbox Account](#)

To obtain Braintree API credentials for production or sandbox:

1. Log into the [Control Panel](#)
2. Navigate to **Account > My user**
3. Under **Authorization**, click **API Keys**
 - a. If no API keys appear, click **Generate New**
4. Click **View** under the **Private Key** column

[Documentation for Important Gateway Credentials](#)

In order to reproduce certain payment responses, you may need to use special test information and amounts. You can find all testing parameters in the links below.

[Documentation for Testing Information](#)

[Documentation for Testing Information and Go-Live](#)